

[Project Title]

Final Project Report

Author One* Author Two* Author Three
[Course Name & Number, Institution]

May 2026

GitHub Repository: github.com/your-org/your-repo

Abstract

TODO: one paragraph summarising the hypothesis, the method, and the main finding.

1 Hypothesis

We investigate [research question]. Specifically, we hypothesize that *[your hypothesis]*. Our experiments show that *[brief summary of result]*.

2 Context

Member contributions.

- **Author One:** TODO (e.g., designed the parallel algorithm, implemented CUDA kernel).
- **Author Two:** TODO (e.g., conducted performance experiments, produced plots).
- **Author Three:** TODO (e.g., literature review, theoretical analysis).

Broader context. This project *[is / is not]* part of a larger research effort. TODO: one sentence about the research group or application domain if applicable.

Applicability to other courses. Variants of this project could be applied in *[e.g., high-performance computing, machine learning systems, etc.]* courses because TODO.

3 Prior Work

Work 1. ? introduced TODO. We build on their approach by TODO.

Work 2. ? proposed TODO. Our implementation differs in that TODO.

Work 3. ? demonstrated TODO. We compare against their results in Section 4.

*Equal contribution.

4 Empirical Methodology

4.1 Problem Setup

We study parallel training of a feed-forward MLP on MNIST with a fixed optimization setup and matched data split across implementations. The primary model for scaling is $784 \rightarrow 256 \rightarrow 128 \rightarrow 64 \rightarrow 10$, trained with SGD and cross-entropy. The project compares four execution modes:

- **Serial**: single-process baseline.
- **Flat MPI Data Parallel (DP)**: each rank computes local gradients, then performs gradient all-reduce every step.
- **Hierarchical MPI DP**: per-step gradient synchronization split into intra-node reduce, inter-node all-reduce (leaders), and intra-node broadcast.
- **Local SGD**: each rank performs local SGD and averages weights every K steps.

Model-parallel MPI is implemented as a separate backend and is evaluated separately because its decomposition objective differs from DP scaling.

4.2 Sequential Baseline

The sequential baseline uses the same MLP, loss, and optimizer as all MPI variants. It runs a shuffled mini-batch loop over training data and reports `epoch_time_ms` as *training-loop-only* wall time (validation evaluation is measured but excluded from epoch timing). This timing definition is intentionally mirrored in the MPI backends to keep scaling comparisons consistent.

4.3 Parallel Design

Why multiple algorithms are shown. The implementations do not share one execution pattern. Serial training has no communication. Data-parallel methods synchronize gradients or weights with collectives. Model parallelism partitions layers and exchanges intermediate tensors point-to-point. For clarity, we therefore present separate algorithms for (1) serial training, (2) data-parallel training, and (3) model-parallel training.

Serial. Single-process SGD over shuffled mini-batches; no MPI calls.

Algorithm 1 Serial training epoch

Require: shuffled indices I , batch size B , parameters θ

Ensure: updated θ after one epoch

```
1: for  $t = 0, B, 2B, \dots$  do
2:   gather batch  $(x, y)$  from  $I$ 
3:    $g \leftarrow \nabla \ell(\theta; x, y)$ 
4:    $\theta \leftarrow \theta - \eta g$ 
5: end for
```

Flat DP. Each rank processes a disjoint shard of each global batch, computes local gradients, then uses `MPI_Allreduce` on layer gradients to form the global mean gradient before applying the update.

Hierarchical DP. Gradient synchronization is decomposed into three phases: `MPI_Reduce` within a node, `MPI_Allreduce` across node leaders, then `MPI_Bcast` within each node. This reduces inter-node participants but introduces extra synchronization phases.

Local SGD. Ranks execute local updates independently and synchronize model weights via one fused `MPI_Allreduce` every K steps ($K \in \{10, 50\}$ in this report). This trades statistical freshness for lower communication frequency.

Model Parallel (MP). Layers are partitioned across ranks. Forward activations and backward gradients are exchanged via point-to-point MPI between adjacent layer partitions.

Algorithm 2 Data-parallel epoch (flat DP / hierarchical DP / local SGD)

Require: shuffled index list I , global batch size B , rank count P

Ensure: updated model parameters after one epoch

```

1: for global batch offset  $t = 0, B, 2B, \dots$  do
2:   gather local mini-batch  $(x_r, y_r)$  from  $I$ 
3:   compute local gradients  $g_r \leftarrow \nabla \ell(\theta; x_r, y_r)$ 
4:   if mode is Flat DP then
5:      $g \leftarrow \text{AllreduceMean}(g_r)$ 
6:   else if mode is Hierarchical DP then
7:      $g \leftarrow \text{IntraReduce} \rightarrow \text{InterAllreduce} \rightarrow \text{IntraBcast}$ 
8:   else if mode is Local SGD then
9:     set  $g \leftarrow g_r$ ; synchronize weights only every  $K$  steps
10:  end if
11:  apply SGD update with synchronized quantity (gradient or weight)
12: end for
```

Algorithm 3 Model-parallel epoch (layer-partitioned)

Require: layer partition per rank, batch size B

Ensure: updated local layer parameters on each rank

```
1: for each batch  $(x, y)$  do
2:   if rank is first then
3:     compute first local forward block from  $x$ 
4:     send activation to next rank
5:   else
6:     receive activation from previous rank
7:     compute local forward block
8:     if not last rank then send activation to next rank
9:     end if
10:  end if
11:  if rank is last then
12:    compute loss and output gradient from  $(y)$ 
13:    backprop local layers; send gradient to previous rank
14:  else
15:    receive gradient from next rank
16:    backprop local layers;
17:    if not first rank then send gradient to previous rank
18:    end if
19:  end if
20:  apply local SGD update to local layers
21: end for
```

4.4 Implementation Details

The system is implemented in C++17 with MPI collectives/point-to-point communication and BLAS-backed dense linear algebra. Key implementation choices that affect performance and reproducibility are:

- **BLAS acceleration:** matrix multiply and vector updates were moved to BLAS-backed kernels (`sgemm/saxpy/sgemv`) to reduce scalar-loop overhead.
- **Backend isolation:** serial, flat DP, hierarchical DP, local-SGD, and model-parallel paths are built as separate executables with dedicated launch scripts.
- **Memory reuse in hot loops:**
 - preallocated mini-batch feature/label buffers are reused across steps to avoid per-batch allocation churn;
 - gather paths use contiguous writes to improve locality;
 - gradient and workspace buffers are reused in the shared MLP training path.
- **Compiler configuration:** Release builds use `-O3`, with optional `-march=native` and optional IPO/LTO when supported.

Because model parallelism uses a separate `MPLayerSlice` forward/backward path, some MLP-internal workspace optimizations apply directly to serial and data-parallel modes, while MP performance is driven more by partitioning balance and point-to-point communication behavior.

4.5 Experimental Setup

All strong-scaling sweeps were executed through Slurm `srun` launchers. Thread-level oversubscription was disabled by pinning: `OMP_NUM_THREADS=1`, `OPENBLAS_NUM_THREADS=1`, `MKL_NUM_THREADS=1`. The primary sweep used:

- epochs = 10, global batch = 1024
- train/validation samples = 50,000/10,000
- learning rate = 0.03, seed = 42
- hidden layers = 256, 128, 64
- rank ladder: $1n4r \rightarrow 1n16r \rightarrow 1n32r \rightarrow 2n64r \rightarrow 4n128r$

Timing and summary outputs are generated from `results/scaling/*.csv` and `results/scaling/*.log` via `summarize_dp_scaling.py` and `make_scaling_visuals.py`.

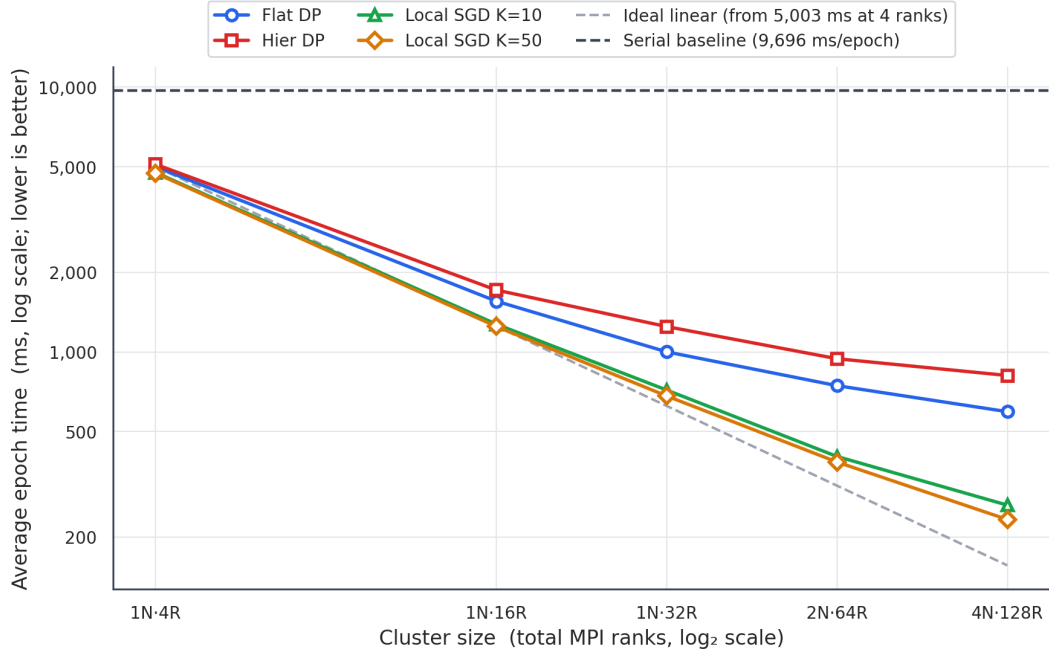
Table 1: Software/runtime configuration used for scaling runs.

Component	Specification
Runtime launcher	Slurm <code>srun</code> + MPI
Threading policy	OMP/BLAS/MKL threads pinned to 1
Build mode	Release (<code>-O3</code> , optional <code>-march=native</code> , optional IPO)
Dataset	MNIST (local cache via <code>prepare_mnist.sh</code>)
Model for main sweep	MLP 256, 128, 64 hidden layers
Scaling configs	$1n4r, 1n16r, 1n32r, 2n64r, 4n128r$

4.6 Results

4.6.1 Scaling Analysis

Figure 1 shows epoch time vs cluster size with serial baseline and ideal scaling reference. Local-SGD achieves the best time at 128 ranks ($K=50$ fastest), while flat DP remains more efficient than hierarchical DP for this model size and workload regime. This indicates that hierarchical communication overhead (extra phases per step) is not amortized sufficiently at the current gradient payload.



N = nodes, R = MPI ranks (e.g. 2N-64R = 2 nodes × 32 ranks each)

Figure 1: Epoch time vs cluster size with serial baseline and ideal line (MLP 256 → 128 → 64, MNIST, 10 epochs).

4.6.2 Strong Scaling

Using flat DP at $1n4r$ as baseline (≈ 5003 ms/epoch), strong-scaling speedup and efficiency are:

Table 2: Strong scaling for flat DP (baseline: $1n4r$).

Config	Time (ms)	Speedup $S(p)$	Efficiency $E(p)$
1n4r	5003	1.00×	100.0%
1n16r	1555	3.22×	80.4%
1n32r	1001	5.00×	62.5%
2n64r	746	6.71×	41.9%
4n128r	595	8.40×	26.3%

Compared to flat DP, hierarchical DP trails at all tested scales in the current runs, while Local-SGD surpasses flat DP speedup as K increases, reaching $21.46\times$ speedup at $4n128r$ ($K=50$) relative to the flat-DP 4-rank baseline.

4.6.3 Weak Scaling

This report focuses on strong scaling. A dedicated weak-scaling sweep (proportionally increasing data size with rank count) is not yet included in the current artifact set and is left as future work.

4.6.4 Additional Results

Timing breakdowns from logs show:

- flat DP: communication cost appears primarily in `allreduce_ms`, while `other_ms` remains non-negligible at high ranks.
- hierarchical DP: cost is split into `intra_reduce_ms`, `inter_allreduce_ms`, and `intra_bcast_ms`, confirming additional synchronization phases.
- local-SGD: `sync_ms` is much smaller than per-step DP comm cost because communication occurs only every K steps.

Accuracy trade-off is modest for the fastest local-SGD settings in these runs: at 128 ranks, $K=50$ is fastest with a small validation-accuracy drop relative to flat DP.

5 Challenges and Obstacles

[Challenge 1]. TODO: describe what you tried, what happened, and your hypothesis for why.

[Challenge 2]. TODO.

[Challenge 3]. TODO.

References